

StegAI

ML-Guided Image Steganography with AES-256 Encryption

Senior Design Project | Computer Science & Engineering

Steganography

Cryptography

Machine Learning

Academic Year 2025–26

Introduction

Problem Statement & Motivation

The Problem

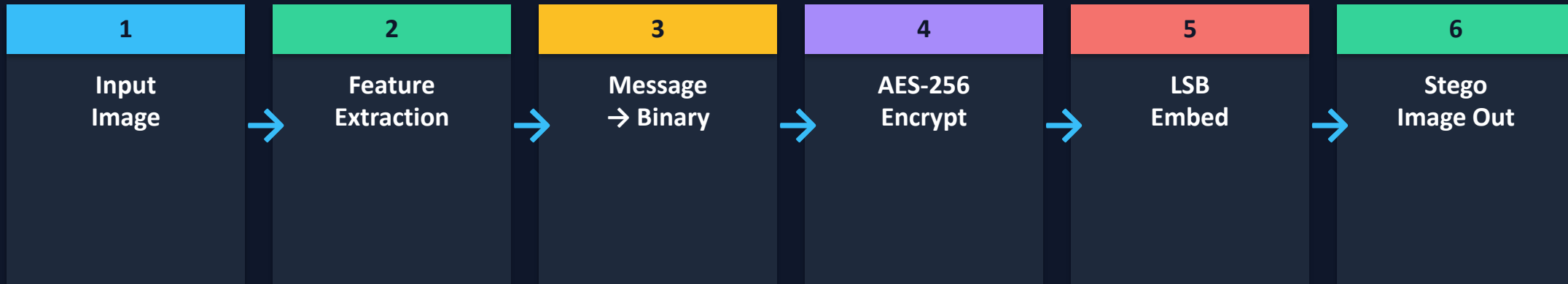
- Digital surveillance & data interception are growing threats
- Encryption alone reveals that secret communication is happening
- Traditional steganography lacks intelligence — embeds in predictable regions
- No standard tool combines ML + Crypto + Steganography in one pipeline

Our Solution — StegAI

- Uses ML (Random Forest) to find complex image regions
- Embeds only in high-texture blocks — harder to detect
- Encrypts data with AES-256 before embedding
- ECC (SECP256R1) used for secure key exchange
- Measures quality with PSNR & SSIM metrics
- Flask web interface — no installation for end-user

System Architecture

End-to-End Workflow



Extraction Path (Reverse):



Quality Metrics (post-embed)

- ▶ PSNR > 40 dB → visually identical images
- ▶ SSIM ≈ 1.0 → structural similarity preserved
- ▶ Histogram comparison shows near-identical pixel distributions

Feature Extraction

ML-Guided Block Selection

Why Feature Extraction?

- ▶ Not all image regions are equal for hiding data
- ▶ Flat/uniform areas → changes are visually obvious
- ▶ Textured/complex areas → changes are invisible to human eye
- ▶ We divide image into 8×8 pixel blocks
- ▶ Each block is scored before embedding

Features Extracted per Block:

- ▶ Standard Deviation — how varied pixel values are
- ▶ Variance — spread of pixel intensities
- ▶ Shannon Entropy — randomness / texture score
- ▶ Block Pre-filter — skips blocks with $\text{Var} < 20$

Random Forest Classifier

Input: 8×8 image block



Feature
Vector: [std, variance]



Random
Forest: 50 decision trees



Output: Safe (1) / Unsafe (0)

Label rule: $\text{std} > 30 \rightarrow \text{Safe block} \checkmark$

Elliptic Curve Cryptography (ECC)

Secure Key Exchange — SECP256R1

What is ECC?

- Public-key cryptosystem
- Based on algebraic structure of elliptic curves over finite fields
- Equation: $y^2 = x^3 + ax + b$
- Security from: Elliptic Curve Discrete Log Problem (ECDLP)
- 256-bit ECC $\approx$ 3072-bit RSA — much smaller keys!

How It Works In StegAI

- Step 1

Generate private key (random scalar)
- Step 2

Derive public key = private_key $\times$ G
- Step 3

ECDH: shared_secret = priv_A $\times$ pub_B
- Step 4

HKDF-SHA256 $\rightarrow$ 32-byte AES key
- Step 5

AES-256 encrypts message with derived key

Why ECC?

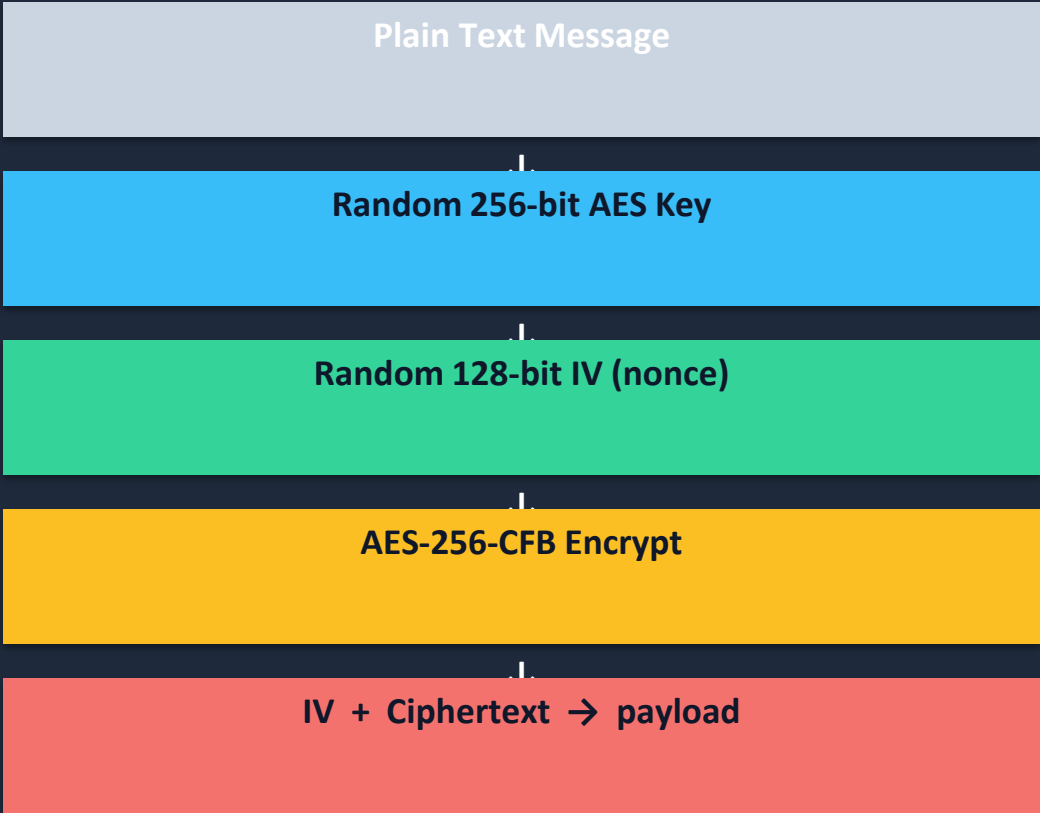
Algorithm	Key Size	Security
RSA	3072 bit	128-bit
ECC	256 bit	128-bit
AES	256 bit	128-bit

- Faster key generation
- Less bandwidth & storage
- Used in TLS, Bitcoin, SSH

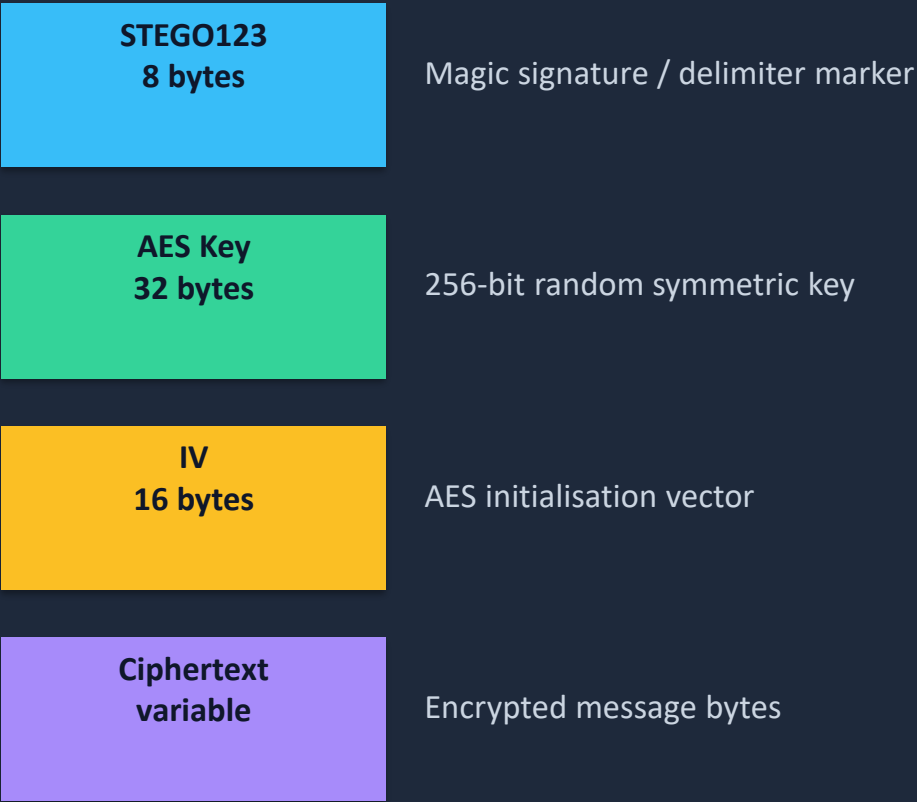
AES-256 Encryption

Symmetric Encryption Before Embedding

Encryption Flow



Payload Structure Embedded



Total overhead per message: 8 + 32 + 16 = 56 bytes before ciphertext

Bit Conversion

Text → ASCII → Binary → Pixel LSBs

Character → Binary

Char		ASCII		Binary (8-bit)
'A'	→	65	→	01000001
'B'	→	66	→	01000010
'Z'	→	90	→	01011010
'a'	→	97	→	01100001
'!'	→	33	→	00100001

LSB Embedding Explained

- ▶ Each pixel channel byte = 8 bits
- ▶ We change only the LAST bit (LSB) per byte
- ▶ Pixel value change: at most ± 1 — invisible to eye!
- ▶ 8 pixel channels needed to store 1 byte of data

Example: embedding bit '1' into pixel 200

Original	200 = 11001000
Mask 0xFE	& 11111110
Embed bit	00000001
Result	201 = 11001001

Core Algorithm

Key Code Snippets

Image Load & Resize (OpenCV)	Binary Conversion
<pre>img = cv2.imread(image_path) img = cv2.resize(img, (512, 512)) flat = img.flatten()</pre>	<pre>binary_str = ''.join(format(byte, '08b') for byte in payload)</pre>
AES Encrypt Call	LSB Embedding (core loop)
<pre>aes_key = os.urandom(32) # 256-bit cipher = encrypt_aes(msg, aes_key) payload = b'STEGO123' + aes_key + cipher</pre>	<pre>for i, bit in enumerate(binary_str): flat[i] = (flat[i] & 0xFE) int(bit)</pre>

Results & Quality Metrics

PSNR · SSIM · Histogram Analysis

PSNR

- ▶ Peak Signal-to-Noise Ratio
- ▶ Measures pixel-level distortion
- ▶ Higher = better quality
- ▶ PSNR > 40 dB = imperceptible change
- ▶ Our system typically: 42–48 dB

SSIM

- ▶ Structural Similarity Index
- ▶ Compares luminance, contrast & structure
- ▶ Range: 0 (totally different) → 1 (identical)
- ▶ SSIM > 0.99 = visually identical
- ▶ Our system typically: 0.996–0.999

Histogram

- ▶ Pixel intensity distribution plot
- ▶ Original vs Stego side-by-side
- ▶ Near-identical curves = no detectable change
- ▶ Steganalysis resistance indicator
- ▶ Auto-generated & shown in web UI

Conclusion & Future Work

Summary and Next Steps

What We Achieved

- ▶ Full embed / extract pipeline working end-to-end
- ▶ ML selects high-texture blocks for safe embedding
- ▶ AES-256-CFB encrypts payload before hiding
- ▶ ECC (SECP256R1) + HKDF for secure key derivation
- ▶ PSNR & SSIM confirm imperceptible image quality
- ▶ Flask web interface — accessible via browser
- ▶ Histogram comparison for steganalysis resistance

Future Improvements

- ▶ Train ML on real steganalysis datasets (e.g. BOSS Base)
- ▶ Add deep learning model (CNN) for block selection
- ▶ Support audio & video steganography
- ▶ Implement full ECC ECDH key exchange between two users
- ▶ Add user authentication & session management
- ▶ Mobile app interface (React Native / Flutter)
- ▶ Publish PSNR/SSIM benchmarks in a research paper

Existing vs Proposed System

What's Different About StegAI?

Existing Approach (Traditional LSB)

- ▶ Basic LSB — data embedded uniformly across all pixels
- ▶ Ignores image texture, complexity, and region suitability
- ▶ Causes predictable statistical patterns in pixel values
- ▶ Easily detected by: Histogram Analysis, Chi-square tests, SPAM/SRM models
- ▶ No encryption — data exposed even if stego image is found
- ▶ No payload control — risk of over-embedding

Proposed System (StegAI)

- ▶ Image divided into 8×8 blocks for analysis
- ▶ Each block evaluated using Entropy + Variance
- ▶ High-texture regions selected as safe embedding zones
- ▶ Smooth/flat regions are avoided entirely
- ▶ Adaptive LSB — embedding locations are controlled
- ▶ AES-256 encrypts message BEFORE hiding it
- ▶ Reduces statistical distortion and steganalysis detectability

Literature Survey

Related Work & Prior Art — Part 1 of 2

SPAM Pevný et al., 2010	Subtractive Pixel Adjacency Matrix to detect stego artifacts via pixel differences	✓ Effective against LSB; low compute cost ✗ Limited against modern adaptive steganography
SRM Fridrich & Kodovský, 2012	Rich Models using high-dimensional residual-based features for steganalysis	✓ Very high detection accuracy; benchmark standard ✗ Large feature space; not suitable for real-time
IStego100K Yang et al., 2023	Large-scale benchmark dataset for training & evaluating steganalysis models	✓ Diverse embedding methods; supports ML training ✗ High storage required; not for lightweight systems

Literature Survey

Related Work & Prior Art — Part 2 of 2

UNIWARD Holub et al., 2014	Universal distortion function for content-adaptive steganography across domains	✓ Reduced detectability; adaptable to multiple domains ✗ Computationally intensive; complex distortion modeling	Research Gap → No system combines ML + Crypto + Adaptive Steg → No web-based real-time pipeline exists → StegAI fills this gap
Survey 2025 Apau et al., 2025	Comprehensive survey of techniques resisting statistical steganalysis attacks	✓ Categorizes adaptive, statistical & content-aware methods ✗ Survey only; no unified algorithm proposed	
DL Survey De La Croix, 2024	Survey on deep learning–based steganalysis — CNN and hybrid techniques	✓ Explains evolution from statistical to deep learning ✗ Detection-focused; requires large datasets	

Threat Model & Attack Analysis

How StegAI Resists Detection

StegAI is designed to resist Low-Level and Mid-Level steganalysis attacks through AI-guided adaptive embedding and controlled payload capacity.

Low-Level Attacks

- LSB Statistical Analysis — detects unnatural LSB randomness
- Histogram Analysis — compares pixel intensity distributions
- Noise Variance Analysis — detects added noise from embedding

Mid-Level Attacks

- Chi-Square Attack — tests pixel pair probability distributions
- SPAM-Based Analysis — checks pixel adjacency & difference patterns
- Feature-Based ML Attacks — SVM/RF using entropy & co-occurrence
- Texture Inconsistency Detection — abnormal texture transitions

StegAI Defence: Selects only high-entropy blocks ▸ Limits payload per region ▸ Encrypts before embedding ▸ Histogram stays natural

Technology Stack

Tools, Libraries & Frameworks Used

Frontend

- HTML5 + CSS3
- JavaScript (Vanilla)
- Jinja2 Templates
- Responsive UI — image preview + download

Backend

- Python 3.x
- Flask (REST API)
- Werkzeug (file security)
- RESTful route handling

Encryption

- cryptography library
- ECC — SECP256R1 curve
- HKDF-SHA256 key derivation
- AES-256-CFB mode

Image Proc.

- OpenCV (cv2)
- NumPy arrays
- LSB steganography
- PSNR / SSIM metrics

ML Module

- scikit-learn
- Random Forest (50 trees)
- Block feature extraction
- std & variance features

Evaluation

- PSNR > 40 dB target
- SSIM $\approx$ 1.0 target
- Matplotlib histogram
- Side-by-side comparison

Modules Description

Five Core Components of StegAI

Image Analysis Module

ecc_module.py

- Divides image into 8×8 pixel blocks
- Computes Shannon entropy per block
- Calculates variance for texture scoring
- Identifies safe embedding regions

ECC Encryption Module

ecc_module.py

- Generates SECP256R1 key pair
- ECDH + HKDF derives shared AES key
- Encrypts message before embedding
- Ensures security even if stego is found

Adaptive Embedding Module

steg_module.py

- Embeds only in ML-approved safe blocks
- Controls bits per pixel dynamically
- LSB embedding with 0xFE mask
- Appends STEGO123 delimiter

Extraction & Decryption

steg_module.py

- Reads LSBs sequentially from image
- Assembles bytes until delimiter found
- Strips AES key and IV from payload
- Decrypts and returns plaintext

Evaluation Module

steg_module.py

- Computes PSNR (target > 40 dB)
- Computes SSIM (target ≈ 0.99+)
- Generates pixel histogram comparison
- Matplotlib — saves to static/hist.png

Implementation Progress

What's Done · What's In Progress

Completed

- ▶ Web-based interface — image upload, message input, preview & download
- ▶ ECC (SECP256R1) encryption and decryption — fully working
- ▶ AES-256-CFB encrypt / decrypt pipeline — verified
- ▶ Basic LSB embedding and extraction — tested on sample images
- ▶ AI feature extraction (entropy + variance) — completed
- ▶ Random Forest ML classifier — trained and integrated
- ▶ PSNR & SSIM metrics — computed and displayed in UI
- ▶ Pixel histogram comparison — auto-generated and shown
- ▶ End-to-end pipeline — tested successfully on multiple images

In Progress

- ▶ Integration of full ECC ECDH two-party key exchange
- ▶ Frequency-domain features (DCT) for ML classifier
- ▶ Dynamic payload capacity control per block
- ▶ Chi-square steganalysis resistance benchmarking
- ▶ Performance testing on large image datasets

Evaluation Strategy

Measuring Quality, Security & Performance



Image Quality

- PSNR — Peak Signal-to-Noise Ratio
- Measures pixel-level distortion after embed
- Target: PSNR > 40 dB (imperceptible change)
- SSIM — Structural Similarity Index
- Compares luminance, contrast & structure
- Target: SSIM > 0.99



Security

- Histogram comparison — original vs stego
- Near-identical curves = low detectability
- Chi-square test — pixel pair randomness
- SPAM feature analysis resistance
- Goal: statistical indistinguishability



Performance

- Payload capacity vs image distortion trade-off
- Accuracy of message extraction (100% target)
- Embedding speed on various image sizes
- Resistance to image processing (compress/resize)
- Benchmarked on standard test images

References

Cited Literature & Resources

[1] Pevný, T., Bas, P., Fridrich, J. (2010). Steganalysis by Subtractive Pixel Adjacency Matrix. IEEE TIFS.

https://ws2.binghamton.edu/fridrich/Research/paper_6_dc.pdf

[2] Fridrich, J., Kodovský, J. (2012). Rich Models for Steganalysis of Digital Images. IEEE TIFS.

<https://dde.binghamton.edu/kodovsky/pdf/TIFS2012-SRM.pdf>

[3] Yang, Z., Zhang, Y., Li, Y. (2023). IStego100K: A Large-Scale Image Steganalysis Dataset. IEEE Access.

<https://ieeexplore.ieee.org/document/6249077>

[4] Holub, V., Fridrich, J., Denemark, T. (2014). Universal Distortion Function for Steganography. IEEE TIFS.

<https://www.researchgate.net/publication/259639875>

[5] Apau, R., Badu, K., Amankwah, R. (2025). Image Steganography Techniques for Resisting Statistical Steganalysis. Scientific Reports.

<https://pmc.ncbi.nlm.nih.gov/articles/PMC11404826/>

[6] De La Croix, N.J. (2024). Comprehensive Survey on Image Steganalysis Using Deep Learning. Results in Engineering.

<https://www.sciencedirect.com/science/article/pii/S2590005624000195>